



Мне бы модель где 12 д состояний пространство и 6д состояний времени вот какую модель

Ты описываешь очень мощную и нестандартную модель:

- 12D пространства состояний (конфигурационное пространство системы),
- и 6D состояния времени — не просто координаты времени, а 6-мерное «временное состояние» (возможно, как эволюция, цикл, скорость, ускорение и т.п.).

Ниже — чистый Python-код HighDimStateSystem, реализующий такую модель максимально универсально.

1. Структура модели

- Пространство состояний $\mathcal{S} \in \mathbb{R}^{12}$: 12-мерный вектор $s = (s_1, \dots, s_{12})$,
- Временное состояние $\tau \in \mathbb{R}^6$: 6-мерный вектор времени/параметров времени,
- полная система живёт в 18-мерном пространстве состояний: $(s_1, \dots, s_{12}, \tau_1, \dots, \tau_6)$.

Мы не будем хранить всё 18D-решёткой (это непрактично), а сделаем динамику векторов состояний:

$$\frac{ds}{dt} = F_s(s, \tau), \quad \frac{d\tau}{dt} = F_\tau(s, \tau),$$

где t — параметр внешнего (мета-)времени.

2. Код: 12D-пространство + 6D-время

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

class HighDimStateSystem:
    """
    Модель высокоразмерного состояния:
     $s \in \mathbb{R}^{12}$  (пространство состояний)
     $\tau \in \mathbb{R}^6$  (временное состояние)

    Полная система живёт в 18D, но динамика:
     $s'(t) = F_s(s, \tau)$ 
     $\tau'(t) = F_\tau(s, \tau)$ 
    """
```

```

"""

def __init__(self, seed=None):
    np.random.seed(seed)

    # размерности
    self.dim_s = 12 # пространство состояний
    self.dim_tau = 6 # временное состояние

    # случайные параметры модели (можно менять)
    self.A_s = np.random.normal(0, 0.5, (self.dim_s, self.dim_s))
    self.B_s = np.random.normal(0, 0.1, (self.dim_s, self.dim_tau))

    self.A_tau = np.random.normal(0, 0.3, (self.dim_tau, self.dim_tau))
    self.B_tau = np.random.normal(0, 0.05, (self.dim_tau, self.dim_s))

def _force_s(self, s, tau):
    """
    Внутренняя сила по s:
        F_s(s, tau) = A_s @ s + B_s @ tau + нелинейный вклад

    В простейшей модели можно линейно, но добавим квадратичный слой.
    """
    linear = self.A_s @ s + self.B_s @ tau
    nonlinear = 0.1 * np.tanh(s) # удерживаем систему в умеренном диапазоне
    return linear + nonlinear

def _force_tau(self, s, tau):
    """
    Внутренняя сила по tau:
        F_tau(s, tau) = A_tau @ tau + B_tau @ s
    """
    return self.A_tau @ tau + self.B_tau @ s

def rhs(self, t, y):
    """
    Правая часть ODE: 18D-вектор:
        y[:12] = s
        y[12:] = tau
    """
    s = y[:self.dim_s]
    tau = y[self.dim_s:]

    ds = self._force_s(s, tau)
    dtau = self._force_tau(s, tau)

    return np.concatenate([ds, dtau])

def simulate(self, s0=None, tau0=None, t_span=(0.0, 20.0), n_steps=1000):
    """
    Решить ODE и вернуть траекторию.
    """
    if s0 is None:
        s0 = np.random.normal(0, 0.1, self.dim_s)
    if tau0 is None:
        tau0 = np.random.normal(0, 0.05, self.dim_tau)

```

```

y0 = np.concatenate([s0, tau0])
t_eval = np.linspace(*t_span, n_steps)

sol = solve_ivp(
    self.rhs,
    t_span,
    y0,
    t_eval=t_eval,
    method='RK45'
)

# разбор решения
s_traj = sol.y[:self.dim_s, :] # 12 x N
tau_traj = sol.y[self.dim_s:, :] # 6 x N

return {
    't': sol.t,
    's': s_traj,
    'tau': tau_traj,
}

def plot_3d_s_slice(self, s_traj, title="3D-срез состояния s(t)":
    """
    Отображаем 3D-срез 12D-вектора s(t) (первые 3 компоненты).
    """
    plt.figure(figsize=(10, 8))
    ax = plt.axes(projection='3d')

    ax.plot(s_traj[0], s_traj[1], s_traj[2], label="s(t)")
    ax.set_xlabel("s1")
    ax.set_ylabel("s2")
    ax.set_zlabel("s3")
    ax.set_title(title)
    ax.legend()
    plt.show()

def plot_tau(self, tau_traj, title="Временные состояния τ(t)":
    """
    Отображаем 6D-τ как 6 временных рядов.
    """
    plt.figure(figsize=(12, 5))
    t = np.arange(tau_traj.shape[1])
    for i in range(self.dim_tau):
        plt.plot(t, tau_traj[i], label=f"τ{i+1}")
    plt.legend()
    plt.title(title)
    plt.xlabel("t")
    plt.ylabel("τ")
    plt.grid(True)
    plt.show()

# ===== ПРИМЕР ИСПОЛЬЗОВАНИЯ =====

if __name__ == "__main__":

```

```

# Создаём систему
sys = HighDimStateSystem(seed=42)

# Симуляция
res = sys.simulate(
    s0=np.random.normal(0, 0.05, 12), # малое возмущение в 12D
    tau0=np.zeros(6),                 # нулевые начальные временные состояния
    t_span=(0.0, 30.0),
    n_steps=1000
)

# 3D-срез траектории s(t)
sys.plot_3d_s_slice(res['s'])

# Временные состояния τ(t)
sys.plot_tau(res['tau'])

```

3. Как это соответствует твоей картинке

- $s \in \mathbb{R}^{12}$ — 12D пространство состояний:
 - геометрия, материальная структура, уровень скрытого/явного рисунка,
 - вложенные слои: геология, океан, культура, технологии, социальное и т.п.
- $\tau \in \mathbb{R}^6$ — 6D временное состояние:
 - можно читать как:
 - 1D — линейное время,
 - 2D+ — циклы, темп, периодичность, «амплитуда исторического цикла»,
 - в простейшей версии — 6 параметров, которые управляют динамикой «повторений» и «переломов».

Модель **общая** и применима ко:

- космическим и геологическим системам,
- социальным и политическим системам,
- биологическим и культурным.

4. Если хочешь углубиться дальше

Могу сделать:

- **физическую интерпретацию осей τ** (например, как скорость, ускорение, цикл подготовки/разрушения),
- или переписать силы в терминах **gradient of energy** (в духе физики):

$$\frac{ds}{dt} = -\nabla_s \mathcal{E}(s, \tau), \quad \frac{d\tau}{dt} = -\nabla_\tau \mathcal{E}(s, \tau),$$

где $\mathcal{E}$ — функция энергии/потенциала.

Скажи, хочешь ли ты «**физическую**» версию (с энергией и градиентами) или **чисто структурную/дискурсивную** (как надстройка над оригами-моделью).